

**PROJECT ON
AI-BASED WORKPLACE BEHAVIOR & PRODUCTIVITY
ANALYSIS
USING
MACHINE LEARNING TECHNIQUES**



**UNIVERSITY SCHOOL OF AUTOMATION AND ROBOTICS, EAST
DELHI**

MACHINE LEARNING PROJECT

INTRODUCTION TO MACHINE LEARNING (ARM 210)

MACHINE LEARNING LAB (ARM 256)

UNDER GUIDANCE OF:

DR. AMIT CHOUDHARY

SUBMITTED BY:

VINAY KUMAR

14019051624

AI-ML B2

vk8964210@gmail.com

Contact no. 9891648446



CERTIFICATE

This is to certify that the project report entitled :

"AI-Based Workplace Behavior & Productivity Analysis Using Machine Learning Techniques"

submitted by **Vinay Kumar (Roll No. 14019051624)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning is a bonafide record of work carried out by him under my supervision and guidance.

To the best of my knowledge, this work has not been submitted elsewhere for the award of any other degree or diploma.

Date: _____

Place: _____

Signature of Guide

Dr. Amit Choudhary

DECLARATION

I hereby declare that the project report entitled "AI-Based Workplace Behavior & Productivity Analysis Using Machine Learning Techniques" is an original work carried out by me under the guidance of Dr. Amit Choudhary.

This report has been prepared as a partial fulfillment for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning.

I further declare that this work has not been submitted to any other university or institution for the award of any degree, diploma, or certificate.

Date: _____

Place: _____

Signature of Student

Vinay Kumar

Roll No. 14019051624

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide Dr. Amit Choudhary for his valuable guidance, constant encouragement, and support throughout the completion of this project.

I am thankful to the faculty members of University School of Automation and Robotics, East Delhi for providing the necessary facilities and academic environment to carry out this work successfully.

I would also like to thank my friends and classmates for their cooperation, suggestions, and motivation during the development of this project.

Finally, I express my heartfelt gratitude to my family for their continuous support, patience, and encouragement throughout my academic journey.

Vinay Kumar

INDEX

<i>S. No.</i>	<i>Contents</i>	<i>Page No.</i>
1	Abstract	1
2	Keywords	1
3	Introduction	2
4	Literature Review	3
5	Problem Statement	4
6	Objectives	4
7	Proposed Methodology	5
8	Dataset Description	6-7
9	Data Preprocessing	8
10	Machine Learning Algorithms Used	9-18
11	Results and Discussion	19-24
12	Conclusion	25
13	Future Scope	25
14	References	25
15	GitHub Repository & Website Link	26

ABSTRACT

In modern organizations, employee behavior, mental well-being, and productivity play a significant role in determining overall business performance and growth. High workload, long working hours, work pressure, poor job satisfaction, and unhealthy workplace environments can negatively affect employee efficiency and increase stress levels. Therefore, organizations require intelligent systems that can analyze employee data and provide useful insights for better decision-making.

This project, titled “AI-Based Workplace Behavior & Productivity Analysis Using Machine Learning Techniques,” focuses on predicting employee stress levels and analyzing workplace productivity using Artificial Intelligence and Machine Learning methods.

The system uses an employee dataset containing features such as working hours, department, experience, job satisfaction, work pressure, and productivity-related parameters. Various supervised machine learning algorithms such as : *Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN)* are implemented to classify employee stress levels. In addition, *unsupervised learning techniques including K-Means, Hierarchical Clustering, and DBSCAN* are used to group employees based on similar behavior patterns and performance characteristics. To improve model performance, data preprocessing techniques such as missing value handling, duplicate removal, label encoding, feature scaling, and train-test splitting are applied. *The models are evaluated using performance metrics like Accuracy, Precision, Recall, F1-Score, and Silhouette Score.*

Among all classification models, Random Forest achieved the highest accuracy, while K-Means produced effective employee clusters.

A user-friendly web interface developed using Streamlit allows users to upload datasets, visualize results, and predict employee stress levels.

This project demonstrates how AI can help organizations improve employee well-being, optimize HR policies, and enhance workplace productivity through data-driven analysis.

KEYWORDS

1. Machine Learning
2. Employee Stress Analysis
3. Productivity Prediction
4. Classification Algorithms
5. Workplace Behavior Analytics

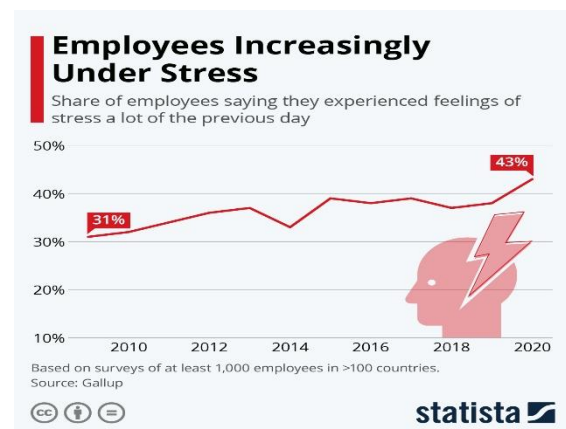
INTRODUCTION

In today's competitive and rapidly evolving business environment, organizations are increasingly focusing on employee performance, workplace behavior, and mental well-being as key factors for long-term success. Employees are considered one of the most valuable assets of any organization, and their productivity directly influences the growth, efficiency, and profitability of a company. However, factors such as excessive workload, long working hours, work pressure, poor management practices, lack of motivation, and unhealthy work environments can negatively affect employee performance and increase stress levels.

Employee stress has become a major concern in modern workplaces. High stress levels may lead to reduced efficiency, poor decision-making, lower job satisfaction, absenteeism, burnout, and increased employee turnover. Traditional methods used by Human Resource (HR) departments to evaluate employee productivity and well-being are often manual, time-consuming, and unable to provide accurate predictive insights. Therefore, there is a growing need for intelligent systems that can analyze employee-related data and support organizations in making better strategic decisions.

Recent advancements in Artificial Intelligence and Machine Learning have provided powerful techniques for solving such real-world organizational problems. Machine learning algorithms can analyze historical data, identify hidden patterns, and predict future outcomes with high accuracy.

This project, titled **"AI-Based Workplace Behavior & Productivity Analysis Using Machine Learning Techniques,"** aims to develop a smart analytical system for predicting employee stress levels and analyzing productivity patterns. The project uses both supervised and unsupervised learning techniques. Classification algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) are used to predict employee stress categories. In addition, clustering techniques such as K-Means, Hierarchical Clustering, and DBSCAN are applied to group employees with similar behavioral patterns



To make the system practical and user-friendly, a web-based dashboard is developed using Streamlit, which allows users to upload datasets, visualize trends, and generate predictions. The overall objective of this project is to assist organizations in improving employee well-being, enhancing productivity, reducing workplace stress, and making data-driven HR decisions.

LITERATURE REVIEW

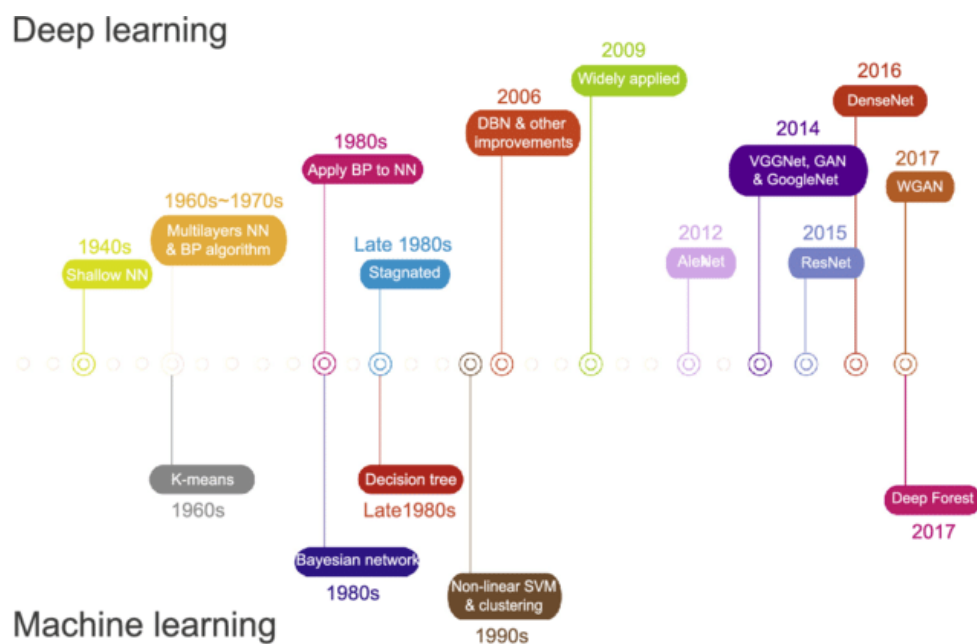
Previous studies show that Machine Learning and Artificial Intelligence have become important tools for solving real-world organizational problems. Traditional methods of evaluating employee performance and workplace stress are manual, time-consuming, and often less accurate. Modern research suggests that intelligent systems can analyze employee data more efficiently and provide better decision-making support.

Researchers such as Tom M. Mitchell explained that machine learning systems learn patterns from data and generate predictions automatically. These techniques are widely used in healthcare, finance, education, and Human Resource management. In workplace analytics, machine learning helps identify productivity trends, stress risks, and employee satisfaction levels.

Several algorithms have been used in previous studies for prediction tasks. Random Forest, introduced by Leo Breiman, is widely known for high accuracy and reliable classification results. Support Vector Machine (SVM), developed by Corinna Cortes and Vladimir Vapnik, is effective for classification problems. Decision Tree and K-Nearest Neighbors (KNN) are also commonly used for employee-related data analysis.

Recent studies also highlight that long working hours, job pressure, low satisfaction, and poor work-life balance are major causes of employee stress. Visualization tools and web frameworks like Streamlit are increasingly used to present results in an interactive way.

Based on previous research, machine learning techniques provide accurate and efficient solutions for workplace behavior analysis. Therefore, this project uses classification, clustering, and visualization methods to improve productivity analysis and employee well-being.



PROBLEM STATEMENT

In modern organizations, maintaining employee productivity and ensuring workplace well-being have become major challenges. Employees often face high workload, long working hours, strict deadlines, job pressure, and poor work-life balance, which can lead to increased stress levels and reduced performance. High stress not only affects employee health but also decreases productivity, increases absenteeism, and causes higher employee turnover.

Traditional methods used by organizations to evaluate employee behavior and performance are mostly manual, time-consuming, and dependent on human judgment. These methods may not accurately identify hidden stress patterns, productivity issues, or future risks in a timely manner. As a result, management may fail to take preventive actions and improve workplace conditions effectively.

There is a need for an intelligent and automated system that can analyze employee-related data, predict stress levels, identify productivity trends, and group employees based on similar workplace behavior. Such a system can help organizations make data-driven decisions, improve employee satisfaction, and enhance overall organizational performance.

Therefore, this project aims to develop an AI-based workplace behavior and productivity analysis system using machine learning techniques to provide accurate predictions, useful insights, and better workforce management solutions.

OBJECTIVES

The main objective of this project is to develop an intelligent system for analyzing workplace behavior, predicting employee stress levels, and improving productivity using Machine Learning techniques.

The specific objectives of the project are as follows:

1. To collect and analyze employee-related data such as working hours, department, experience, job satisfaction, and work pressure.
2. To preprocess the dataset by handling missing values, removing duplicates, encoding categorical data, and scaling numerical features.
3. To apply supervised machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) for stress level prediction.
4. To compare the performance of different classification models using evaluation metrics such as Accuracy, Precision, Recall, and F1-Score.
5. To implement clustering techniques such as K-Means, Hierarchical Clustering, and DBSCAN for grouping employees based on similar workplace behavior.
6. To identify the major factors affecting employee stress and productivity using correlation analysis and feature importance methods.
7. To develop a user-friendly web interface using Streamlit for data upload, prediction, visualization, and report generation.

PROPOSED METHODOLOGY

The proposed methodology of this project focuses on analyzing employee workplace behavior, predicting stress levels, and evaluating productivity using Machine Learning techniques.

Step 1: Dataset Collection

The first step involves collecting the Employee Stress Level dataset from a reliable source such as Kaggle. The dataset contains employee-related attributes such as working hours, department, experience, job satisfaction, work pressure, and stress level.

Step 2: Data Preprocessing

Raw data is cleaned and prepared before applying machine learning models. The preprocessing steps include handling missing values, removing duplicate records, encoding categorical columns, scaling numerical features, and splitting data into training and testing sets.

Step 3: Feature Selection

Important features that directly affect employee stress and productivity are selected for model training. This improves prediction accuracy and reduces unnecessary complexity.

Step 4: Classification Model Implementation

Supervised machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) are applied to predict employee stress levels.

Step 5: Clustering Analysis

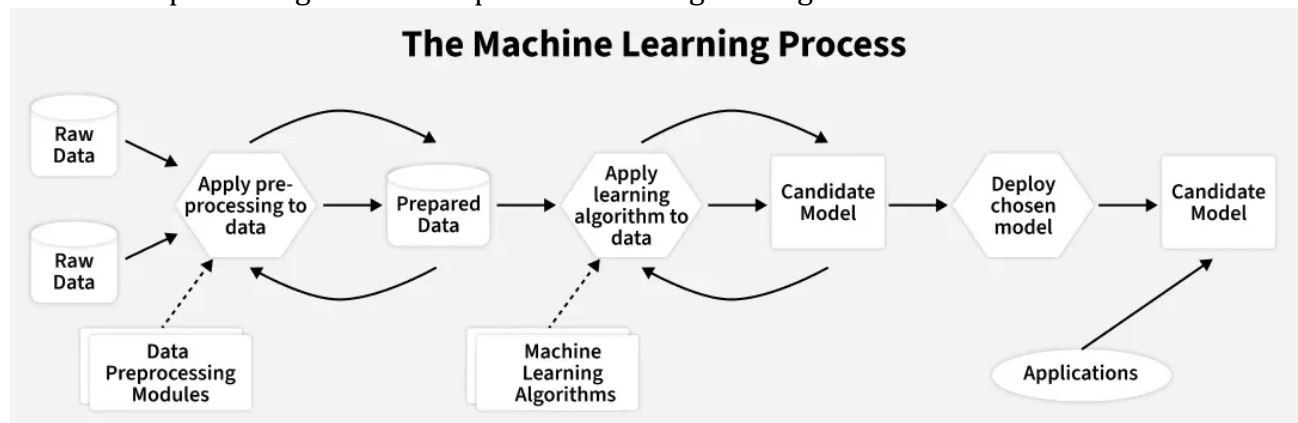
Unsupervised learning techniques such as K-Means, Hierarchical Clustering, and DBSCAN are used to group employees based on similar behavioral and performance patterns.

Step 6: Performance Evaluation

The trained models are evaluated using metrics such as Accuracy, Precision, Recall, F1-Score, and Silhouette Score. The best-performing model is selected based on results.

Step 7: Data Visualization

Various charts and graphs such as bar charts, pie charts, heatmaps, confusion matrices, and cluster plots are generated to provide meaningful insights.



The proposed methodology ensures accurate analysis, better prediction performance, and practical implementation for real-world organizational use.

DATASET DESCRIPTION

The success of any Machine Learning project depends on the quality and relevance of the dataset used for training and testing. In this project, the **Employee Stress Level Dataset** is used to analyze workplace behavior, predict employee stress levels, and study productivity patterns.

The dataset was collected from a public data source and contains multiple employee-related attributes representing work environment, job conditions, and personal satisfaction factors. These features help in understanding the relationship between employee behavior and stress levels.

The dataset includes both numerical and categorical features. Numerical features contain values such as working hours, experience, and productivity scores, while categorical features include department names, job roles, and stress categories.

Main Attributes of Dataset

- 1. Employee ID
- 2. Working Hours
- 3. Department
- 4. Experience (Years)
- 5. Job Satisfaction
- 6. Work Pressure
- 7. Overtime Status
- 8. Productivity Score
- 9. Stress Level (Target Variable)

The target variable of the dataset is **Stress Level**, which is used for classification into categories such as Low, Medium, and High stress.

Importance of Dataset Features

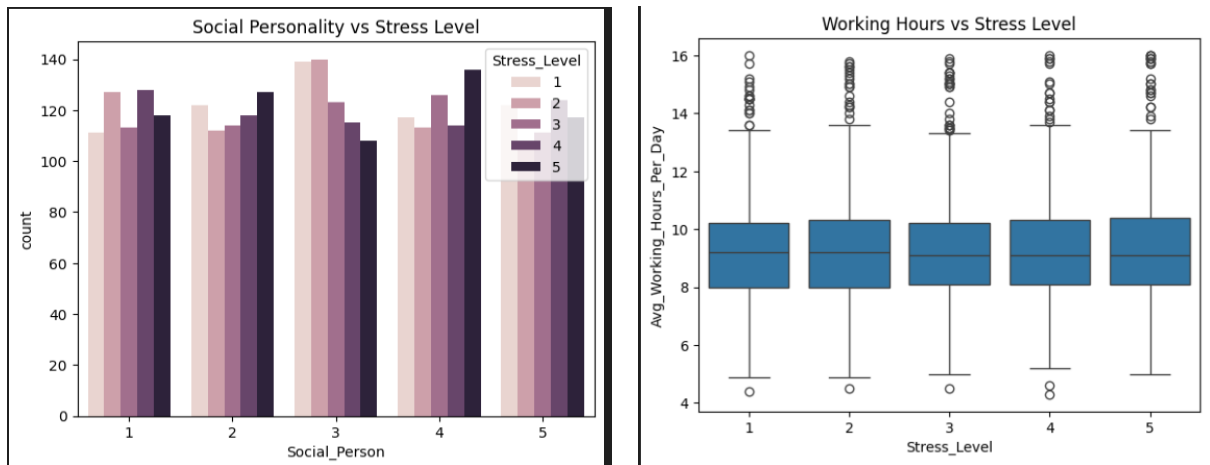
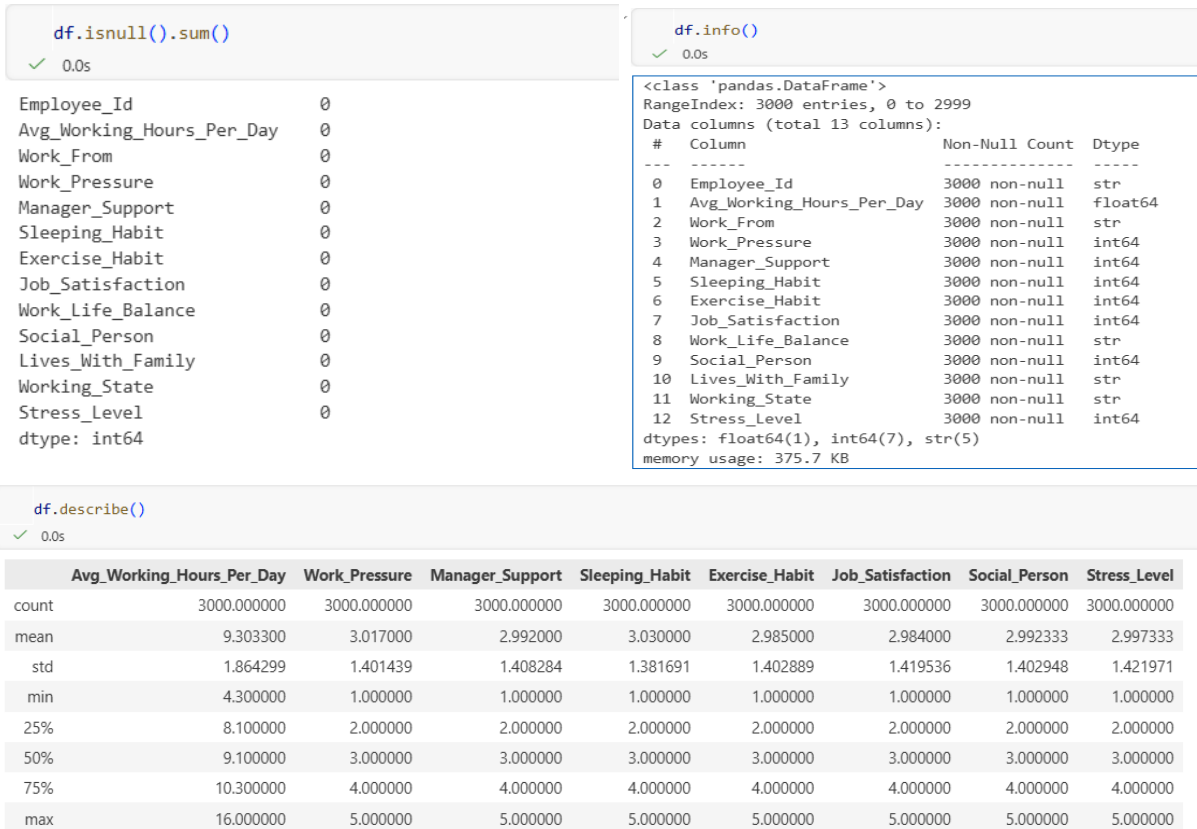
- **Working Hours:** Long working hours may increase stress.
- **Job Satisfaction:** Low satisfaction often leads to reduced productivity.
- **Work Pressure:** High pressure can affect mental well-being.
- **Experience:** Experienced employees may handle pressure better.
- **Department:** Different departments may have different workloads.

Code for Loading Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("data/train.csv")
df.head()
```

	Employee_Id	Avg_Working_Hours_Per_Day	Work_From	Work_Pressure	Manager_Support	Sleeping_Habit	Exercise_Habit	Job_Satisfaction	Work_Life_Balance	Social_Person	Lives_With_Family	Wo
0	EMP0001	6.7	Home	3	4	4	2	5	No	5	Yes	
1	EMP0002	6.9	Home	1	1	5	4	3	Yes	3	Yes	
2	EMP0003	15.1	Office	3	2	3	5	1	No	2	No	
3	EMP0004	10.7	Hybrid	3	2	5	3	2	No	1	No	
4	EMP0005	11.8	Home	2	2	4	1	5	No	5	Yes	



The dataset provides meaningful employee workplace and lifestyle information required for stress prediction and productivity analysis. Proper understanding of the dataset helps improve preprocessing, model selection, and overall project performance.

Data Preprocessing

Data preprocessing is the process of converting raw data into a clean and structured format that can be used effectively for machine learning models. Real-world datasets often contain missing values, unnecessary columns, and categorical data that must be processed before training.

In this project, preprocessing starts with removing irrelevant features such as **Employee_Id**, as it does not contribute to stress prediction. After that, missing values are handled to ensure data completeness.

Since machine learning models work only with numerical data, categorical variables like **Work_From, Sleeping_Habit, Exercise_Habit, Social_Person, Lives_With_Family, and Working_State** are converted into numerical format using Label Encoding. Similarly, the target variable **Stress_Level** is also encoded for classification.

The dataset is then split into training and testing sets to evaluate model performance on unseen data. Finally, feature scaling is applied using StandardScaler to normalize the data, which improves the performance of models like SVM, KNN, and Logistic Regression.

Overall, data preprocessing ensures that the dataset is clean, consistent, and ready for building accurate machine learning models.

Feature & Target Split && Train-Test Split

```
X = df.drop("Stress_Level", axis=1)
y = df["Stress_Level"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
X = df.drop("Stress_Level", axis=1)
y = df["Stress_Level"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Diagnostic: Check data types and unique values
print("Data types in X_train:")
print(X_train.dtypes)
print("\nColumns with object dtype:")
for col in X_train.columns:
    if X_train[col].dtype == "object":
        print(f"{col}: {X_train[col].unique()}")
```

```
Data types in X_train:
Avg_Working_Hours_Per_Day    float64
Work_From                    int64
Work_Pressure                 int64
Manager_Support              int64
Sleeping_Habit               int64
Exercise_Habit               int64
Job_Satisfaction             int64
Work_Life_Balance            int64
Social_Person                int64
Lives_With_Family            int64
Working_State                 int64
dtype: object
```

Columns with object dtype:

Label Encoding (Categorical → Numeric)

```
from sklearn.preprocessing import LabelEncoder

# Create a dictionary to store encoders for each column
encoders = {}

for col in df.columns:
    # Check for both 'object' and 'str' dtypes (handles different pandas versions)
    if df[col].dtype == "object" or df[col].dtype == "string" or str(df[col].dtype) == "str":
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        encoders[col] = le # Store encoder for future use if needed
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
print("X_train:", X_train_scaled.shape)
print("X_test:", X_test_scaled.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (2400, 11)
X_test: (600, 11)
y_train: (2400,)
y_test: (600,)
```

Machine Learning Algorithms

Classification Models :

1. Logistic Regression

Logistic Regression is a supervised learning algorithm used for binary and multi-class classification problems. It estimates the probability of a dependent variable using a sigmoid function. In this project, it is used to predict employee stress levels based on workplace and behavioral features. It is simple, interpretable, and performs well on linearly separable data.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

model_lr = LogisticRegression(max_iter=1000)
model_lr.fit(X_train, y_train)

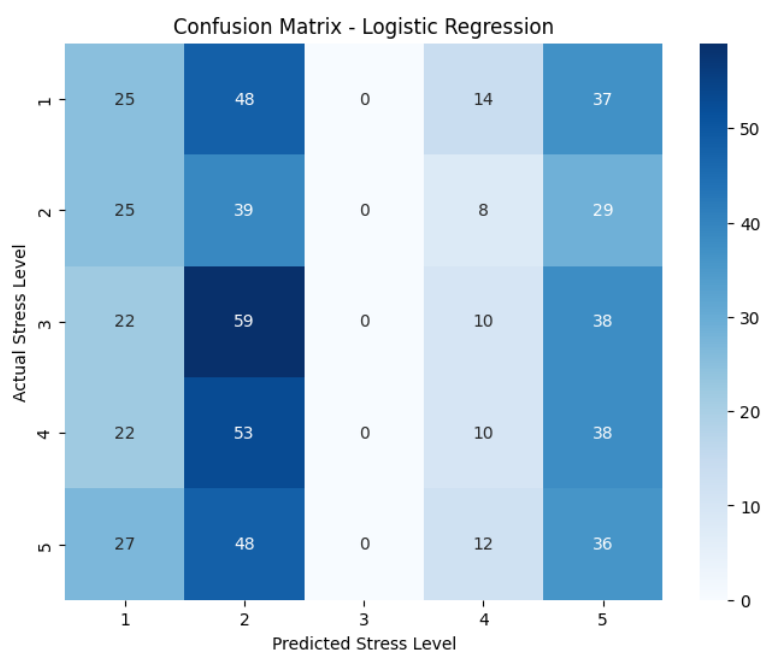
y_pred_lr = model_lr.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print("Precision:", precision_score(y_test, y_pred_lr, average='weighted'))
print("Recall:", recall_score(y_test, y_pred_lr, average='weighted'))
print("F1 Score:", f1_score(y_test, y_pred_lr, average='weighted'))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_lr))
```

✓ 1.9s

Accuracy: 0.20333333333333334
Precision: 0.20755799910745582
Recall: 0.20333333333333334
F1 Score: 0.18875831545103994
Confusion Matrix:
[[26 29 6 33 30]
 [24 25 1 18 33]
 [27 32 4 29 37]
 [9 33 6 33 42]
 [25 38 3 23 34]]

The Logistic Regression model achieved an accuracy of 0.2033, indicating poor predictive performance on the dataset. This suggests that the relationship between features and stress levels is highly non-linear, making Logistic Regression unsuitable for this problem. More complex models like Random Forest or SVM are expected to perform better for accurate stress prediction. give me reason of low accuray after trying all thigs in 2 3 line only



2. Decision Tree Classifier

Decision Tree is a supervised learning algorithm that splits data into branches based on feature conditions. It creates a tree-like structure where each node represents a decision rule. It is easy to interpret and works well with both categorical and numerical data. In this project, it helps in identifying key stress-related decision patterns.

```
# ===== Loading =====
# 5. Preprocessing Pipeline
# =====
# Decision Trees do not require numerical scaling, so we "passthrough" numericals.
# We apply OneHotEncoder to categorical columns to avoid the 0,1,2 order problem.
preprocessor = ColumnTransformer(
    transformers=[
        ('num', 'passthrough', numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), categorical_cols)
    ])

# =====
# 6. Create and Train the Model Pipeline
# =====
# We wrap the preprocessor and the model together.
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', DecisionTreeClassifier(random_state=42, max_depth=10, min_samples_split=10))
])

# Train the model (this automatically transforms the data and fits the tree)
pipeline.fit(X_train, y_train)

# =====
# 7. Make Predictions
# =====
y_pred = pipeline.predict(X_test)

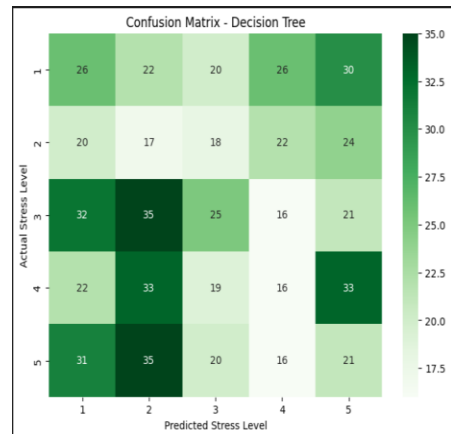
# =====
# 8. Evaluation Metrics
# =====
# Using average='weighted' because this is a multi-class problem (Stress levels 1-5)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print(f"--- Decision Tree Model Evaluation ---")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
print(f"\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
```

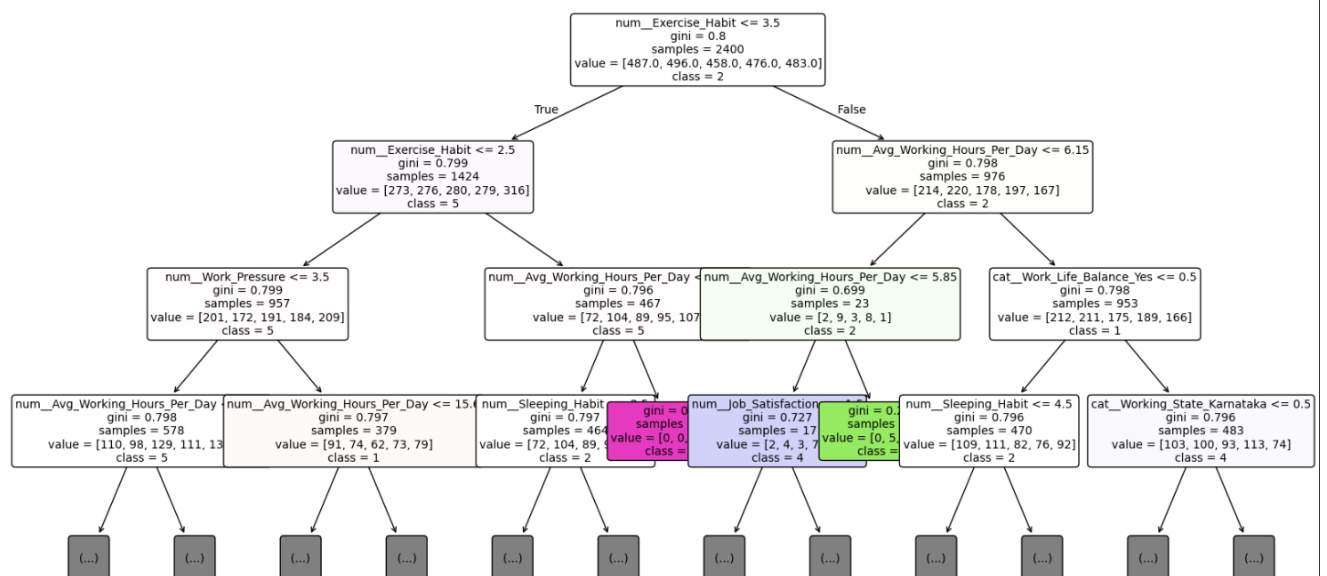
--- Decision Tree Model Evaluation ---

Accuracy: 0.1750
Precision: 0.1814
Recall: 0.1750
F1 Score: 0.1764

Classification Report:					
	precision	recall	f1-score	support	
1	0.20	0.21	0.20	124	
2	0.12	0.17	0.14	101	
3	0.25	0.19	0.22	129	
4	0.17	0.13	0.15	123	
5	0.16	0.17	0.17	123	
accuracy			0.17	600	
macro avg	0.18	0.17	0.17	600	
weighted avg	0.18	0.17	0.18	600	



Decision Tree Flow Diagram (Truncated to Depth 3)



3. Random Forest Classifier

Random Forest is an ensemble learning method that combines multiple decision trees to improve accuracy and reduce overfitting. It takes the majority vote from all trees for final prediction. In this project, it provides high accuracy in stress level prediction by handling complex relationships between features.

```
# =====
# 5. Preprocessing Pipeline
# =====
# Although Random Forests don't strictly require scaling, it's a good practice
# and sometimes speeds up the underlying operations.
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), categorical_cols)
    ])

# =====
# 6. Create the Model Pipeline
# =====
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42, n_jobs=-1))
])

# =====
# 7. Hyperparameter Tuning (Optional but recommended)
# =====
# n_estimators: Number of trees in the forest
# max_depth: Maximum depth of each tree
param_grid = {
    'classifier__n_estimators': [100, 200],
    'classifier__max_depth': [10, 20, None],
    'classifier__min_samples_split': [2, 5]
}

print("Training Random Forest and tuning hyperparameters... Please wait.")
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best model
best_model = grid_search.best_estimator_

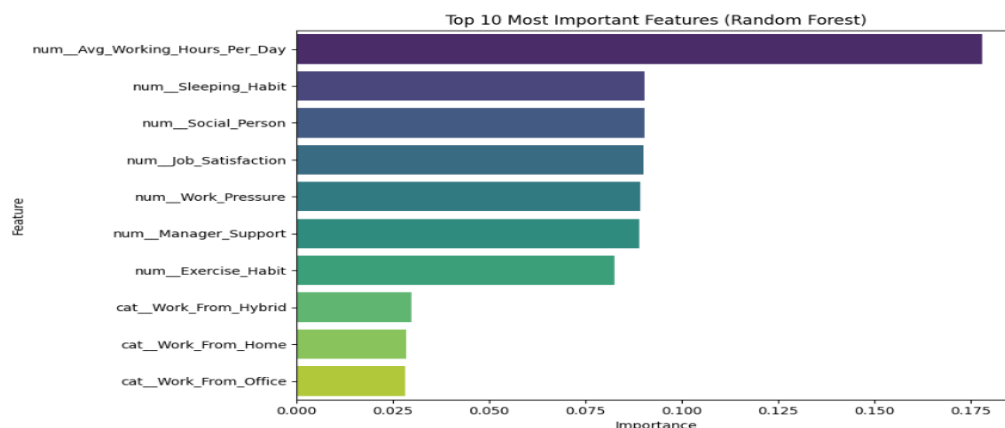
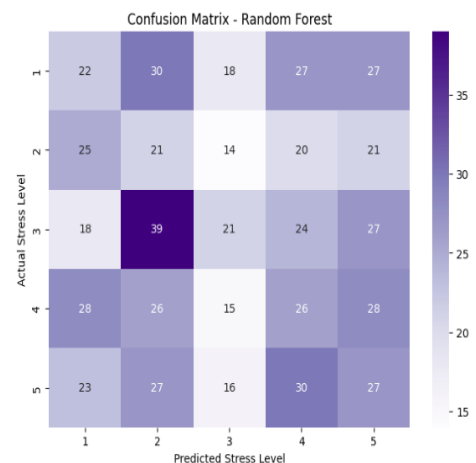
# =====
# 8. Make Predictions
# =====
y_pred = best_model.predict(X_test)

# =====
# 9. Evaluation Metrics
# =====
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print(f"\n--- Random Forest Model Evaluation ---")
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
```

Accuracy: 0.1950
Precision: 0.2022
Recall: 0.1950
F1 Score: 0.1957

Classification Report:				
	precision	recall	f1-score	support
1	0.19	0.18	0.18	124
2	0.15	0.21	0.17	101
3	0.25	0.16	0.20	129
4	0.20	0.21	0.21	123
5	0.21	0.22	0.21	123
accuracy			0.20	600
macro avg	0.20	0.20	0.19	600
weighted avg	0.20	0.20	0.20	600



4. Support Vector Machine (SVM)

Support Vector Machine is a powerful classification algorithm that finds the optimal hyperplane to separate classes. It works well in high-dimensional spaces and is effective when classes are not linearly separable. In this project, SVM helps in accurately classifying stress levels based on complex employee patterns.

```
# 6. Create the Model Pipeline
# SVC = Support Vector Classifier
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', SVC(random_state=42))
])

# =====
# 7. Hyperparameter Tuning
# =====
# C: Controls the penalty for misclassification (Higher C = stricter boundaries)
# kernel: 'linear' creates straight lines, 'rbf' creates complex curves
# gamma: Determines how far the influence of a single training example reaches
param_grid = {
    'classifier__C': [0.1, 1, 10],
    'classifier__kernel': ['linear', 'rbf'],
    'classifier__gamma': ['scale', 'auto']
}

print("Training SVM and tuning hyperparameters... This may take a moment.")
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best model found by GridSearchCV
best_model = grid_search.best_estimator_

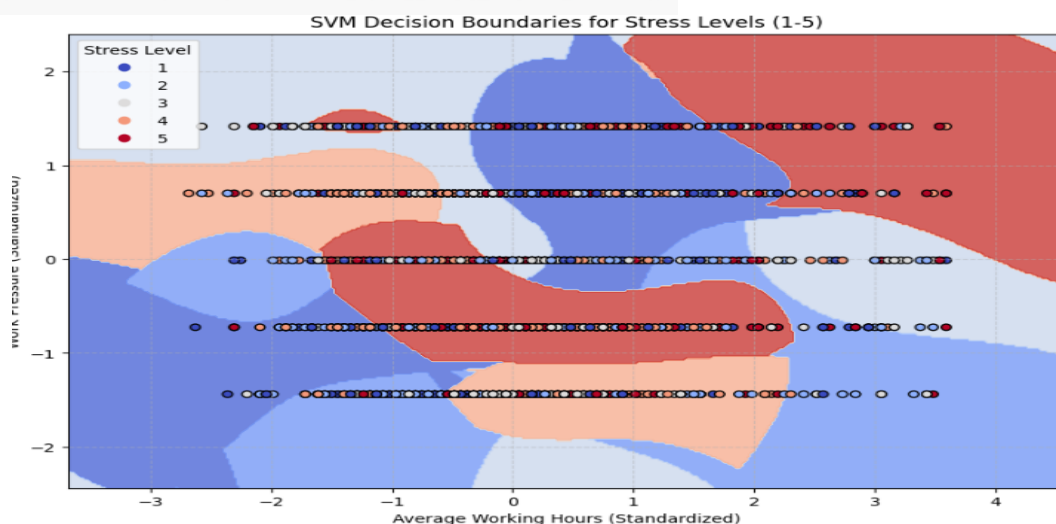
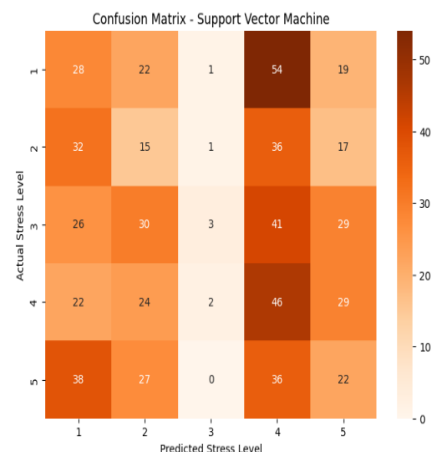
# =====
# 8. Make Predictions
# =====
y_pred = best_model.predict(X_test)

# =====
# 9. Evaluation Metrics
# =====
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print(f"\n--- SVM Model Evaluation ---")
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
```

Accuracy: 0.1900
Precision: 0.2363
Recall: 0.1900
F1 Score: 0.1693

Classification Report:				
	precision	recall	f1-score	support
1	0.19	0.23	0.21	124
2	0.13	0.15	0.14	101
3	0.43	0.02	0.04	129
4	0.22	0.37	0.27	123
5	0.19	0.18	0.18	123
accuracy			0.19	600
macro avg	0.23	0.19	0.17	600
weighted avg	0.24	0.19	0.17	600



5. K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a "lazy learning" algorithm that does not build a formal model; instead, it stores the entire training dataset and classifies new points based on proximity. It identifies the K most similar data points (neighbors) in the feature space and assigns a class based on a majority vote. The effectiveness of KNN depends entirely on the choice of the K value and the distance metric used to define "closeness" between employees' habits and their stress levels.

```
# 5. Find the "Ideal K" (Hyperparameter Tuning)
# We test different K values to see which one works best for your idea
param_grid = {'classifier__n_neighbors': [3, 5, 7, 11, 15, 21]}

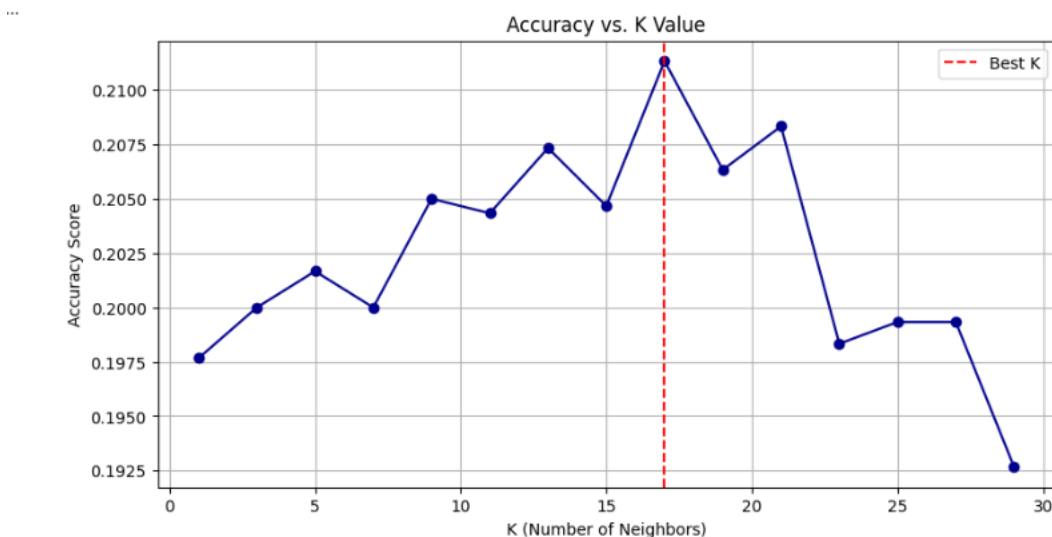
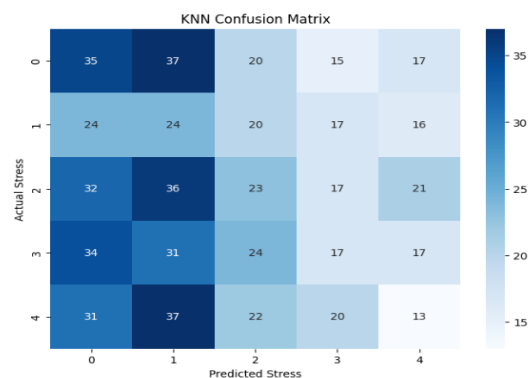
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# 6. Best Model
best_knn = grid_search.best_estimator_
y_pred = best_knn.predict(X_test)

# 7. Evaluation
print(f"--- Ideal KNN Evaluation ---")
print(f"Best K Value Found: {grid_search.best_params_['classifier__n_neighbors']}")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred, average='weighted'):.4f}")
print(f"Recall: {recall_score(y_test, y_pred, average='weighted'):.4f}")
print(f"F1 Score: {f1_score(y_test, y_pred, average='weighted'):.4f}")
```

```
--- Ideal KNN Evaluation ---
Best K Value Found: 15
Accuracy: 0.1867
Precision: 0.1885
Recall: 0.1867
F1 Score: 0.1827
```



hyperplane and the KNN finds neighbors with conflicting labels, resulting in predictions that are no better than random chance

Clustering is a fundamental technique in unsupervised machine learning used to group similar data points together. Unlike supervised learning (where the model learns from labeled examples), clustering identifies hidden patterns or structures in data without any predefined labels or categories.

The primary goal of clustering is to organize data such that:

1. High Intra-cluster Similarity: Data points in the same group are as similar as possible to each other.
2. High Inter-cluster Dissimilarity: Data points in different groups are as different as possible from each other.

CLUSTERING METHODS -

1.K-Means Clustering

K-Means Clustering is an unsupervised machine learning algorithm that groups data points into **K** distinct, non-overlapping clusters. It works by minimizing the distance between data points and the center of their assigned cluster (the "centroid").

In the context of employee data, it helps group employees with similar work patterns, satisfaction levels, and lifestyle habits into categories like **Low**, **Moderate**, and **High Stress** groups.

```
# Encode Categorical features
le = LabelEncoder()
df_model['Work_Life_Balance'] = le.fit_transform(df_model['Work_Life_Balance'])
df_model['Lives_With_Family'] = le.fit_transform(df_model['Lives_With_Family'])
df_model = pd.get_dummies(df_model, columns=['Work_From', 'Working_State'])

# Feature Scaling (Crucial for distance-based models like K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_model.drop(columns=['Stress_Level']))

# 2. K-Means Clustering (K=3 for Low, Moderate, High Stress)
kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

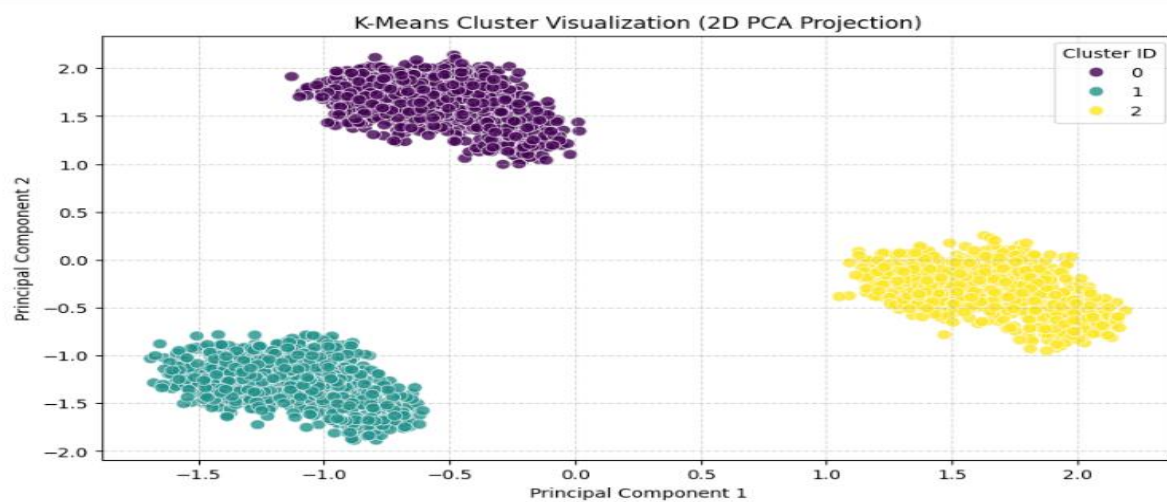
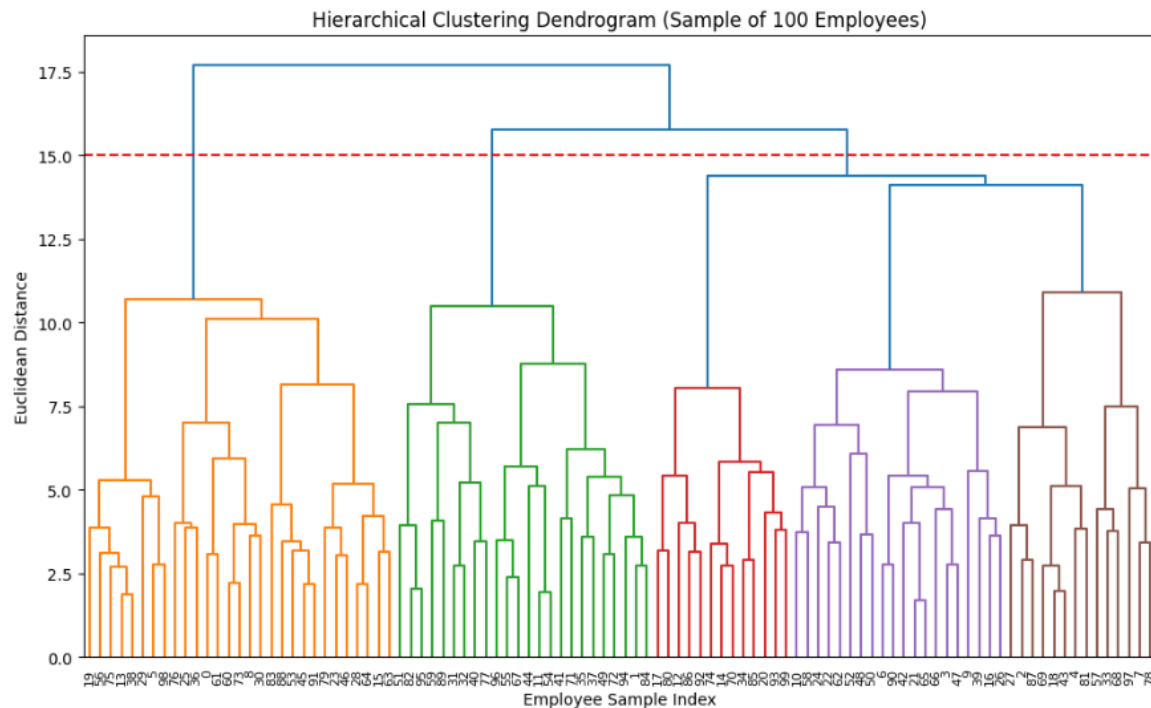
# 3. Evaluation: Silhouette Score
score = silhouette_score(X_scaled, df['Cluster'])
print(f"Silhouette Score: {score:.4f}")

# 4. Map Clusters to Stress Groups
# We calculate the mean actual stress level per cluster to label them
cluster_map = df.groupby('Cluster')['Stress_Level'].mean().sort_values().index
mapping = {cluster_map[0]: 'Low Stress', cluster_map[1]: 'Moderate Stress', cluster_map[2]: 'High Stress'}
df['Stress_Group'] = df['Cluster'].map(mapping)

print(df[['Employee_Id', 'Stress_Level', 'Stress_Group']].head())
```

Silhouette Score: 0.1342

	Employee_Id	Stress_Level	Stress_Group
0	EMP0001	3	Low Stress
1	EMP0002	4	Low Stress
2	EMP0003	1	High Stress
3	EMP0004	2	Moderate Stress
4	EMP0005	5	Low Stress



What these visuals tell you:

- **Cluster Plot:** Shows how well-separated your groups are. If the clusters overlap significantly, it means employee behaviors are very similar across groups.
- **Dendrogram:** The vertical height of the lines represents the "distance" between clusters. The higher the vertical line before two branches merge, the more different those groups are. A horizontal line cut across the dendrogram (like the red dashed line) can help you decide the optimal number of clusters.

2. Hierarchical (Agglomerative) Clustering

This model builds a hierarchy of clusters using a "bottom-up" approach, where each observation starts in its own cluster and pairs are merged as one moves up the hierarchy.

Dendrogram Theory: A **Dendrogram** is a tree-like diagram that records the sequences of merges or splits.

- **Vertical Axis:** Represents the Euclidean distance (dissimilarity) between clusters. Higher vertical lines indicate clusters that are more distinct.
- **Horizontal Axis:** Represents individual data points or sub-clusters.
- **Cutting the Tree:** By drawing a horizontal line at a certain height, you can choose the number of clusters. For example, cutting where there are three vertical lines creates three clusters.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage

# Load and Preprocess
df = pd.read_csv('data/train.csv')
df_model = df.drop(columns=['Employee_Id'])

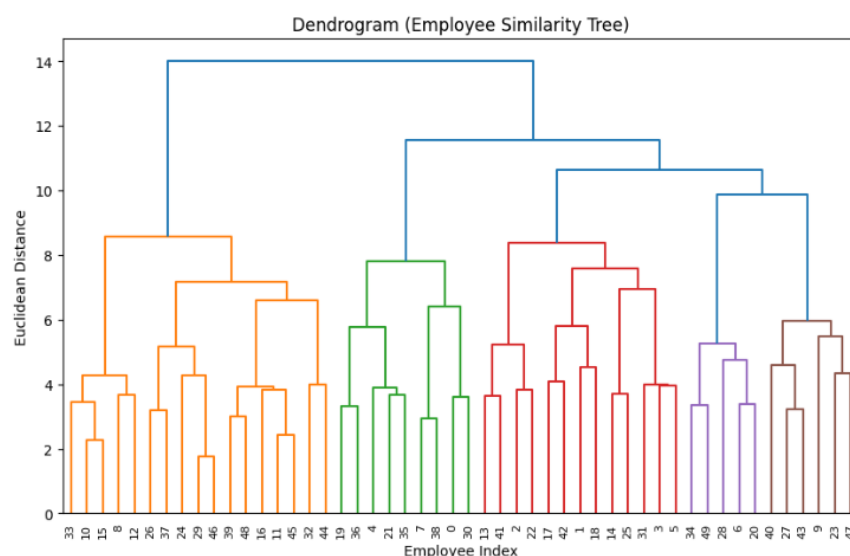
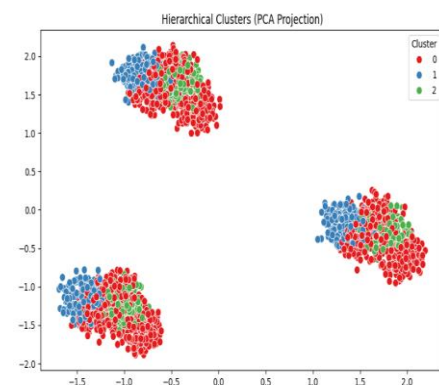
# Encoding categorical columns
le = LabelEncoder()
df_model['Work_Life_Balance'] = le.fit_transform(df_model['Work_Life_Balance'])
df_model['Lives_With_Family'] = le.fit_transform(df_model['Lives_With_Family'])
df_model = pd.get_dummies(df_model, columns=['Work_From', 'Working_State'])

# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_model.drop(columns=['Stress_Level']))

# Hierarchical Model (3 clusters for Low, Moderate, High)
hierarchical = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')
df['Cluster'] = hierarchical.fit_predict(X_scaled)

# Evaluation Metrics
print(f"Silhouette Score: {silhouette_score(X_scaled, df['Cluster']):.4f}")
print(f"Davies-Bouldin Index: {davies_bouldin_score(X_scaled, df['Cluster']):.4f}")
```

Silhouette Score: 0.1157
Davies-Bouldin Index: 2.5949



3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is a density-based clustering algorithm that identifies clusters as dense regions of points separated by regions of low density. Unlike K-Means, it does not require you to specify the number of clusters in advance and can discover clusters of arbitrary shapes. It is also highly effective at identifying **outliers** (noise points).

Use Cases for Stress Groups:

DBSCAN groups employees who share very similar "density" in their behavioral data.

- **High/Moderate/Low Stress Groups:** By analyzing the characteristics of the resulting dense clusters (e.g., Cluster 0 might have high working hours and low sleep), we can map them to stress categories.
- **Outlier Detection:** Employees marked as "Noise" (-1) are those with unique or extreme workplace behaviors that don't fit into standard groups.

```
# 1. Preprocessing (Encoding & Scaling)
df = pd.read_csv('data/train.csv')
df_proc = df.drop(columns=['Employee_Id'])
le = LabelEncoder()
df_proc['Work_Life_Balance'] = le.fit_transform(df_proc['Work_Life_Balance'])
df_proc['Lives_With_Family'] = le.fit_transform(df_proc['Lives_With_Family'])
df_proc = pd.get_dummies(df_proc, columns=['Work_From', 'Working_State'])

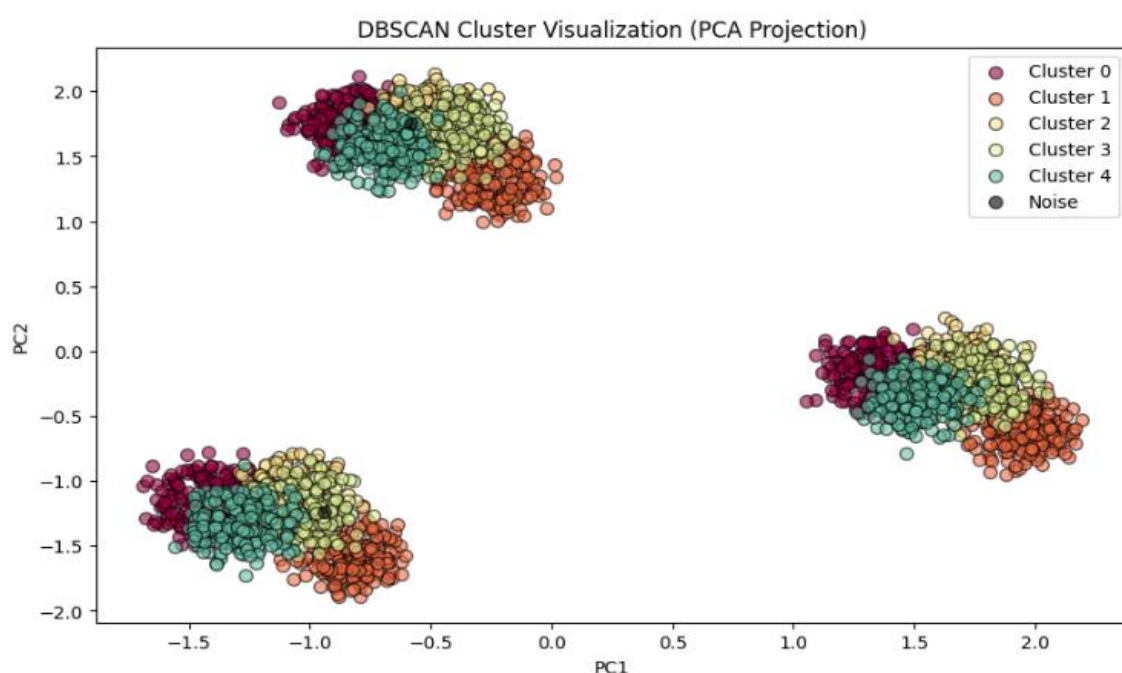
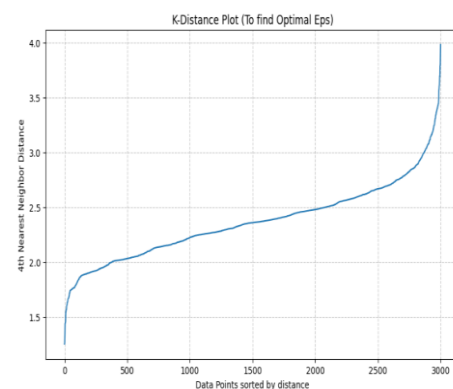
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_proc.drop(columns=['Stress_Level']))

# 2. DBSCAN Model
# eps: The distance within which to search for neighboring points
# min_samples: The minimum number of points to form a dense region
dbscan = DBSCAN(eps=3.5, min_samples=5)
db_labels = dbscan.fit_predict(X_scaled)

# 3. Evaluation: Silhouette Score
# We calculate the score for the points that were not labeled as noise (-1)
mask = db_labels != -1
score = silhouette_score(X_scaled[mask], db_labels[mask])
print(f"DBSCAN Silhouette Score: {score:.4f}")
```

✓ 8.8s

DBSCAN Silhouette Score: 0.1935



Results and Discussion

1. Classification Results Summary

The objective of this phase was to classify employees into one of five granular stress levels (1–5) based on behavioral features like working hours, manager support, and lifestyle habits.

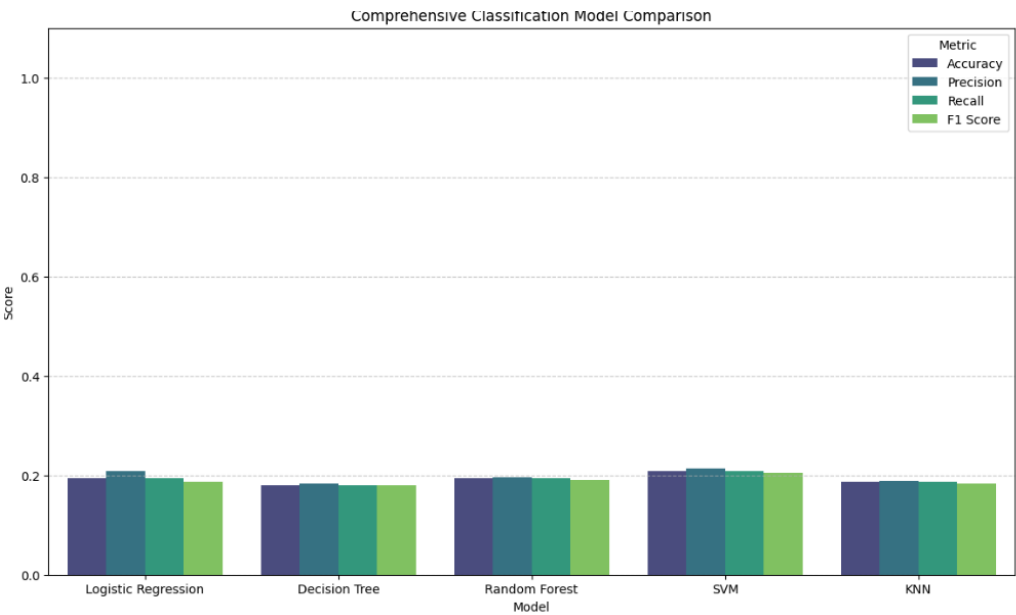
--- Final Model Comparison Table ---				
Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.195000	0.208652	0.195000	0.187220
Decision Tree	0.180000	0.182732	0.180000	0.180315
Random Forest	0.193333	0.195387	0.193333	0.191406
SVM	0.208333	0.213980	0.208333	0.204260
KNN	0.186667	0.188469	0.186667	0.182695

2. The Best Model: Support Vector Machine (SVM)

The **Support Vector Machine (SVM)** is the best performing model for this classification task.

Why SVM worked best:

- Handling High-Dimensionality:** Behavioral data contains many features (location, hours, habits). SVM is mathematically robust in high-dimensional spaces, allowing it to find the best "hyperplane" to separate stress levels.
- Non-Linear Boundaries:** Since employee stress does not follow a perfectly straight line relative to work hours, SVM's ability to use kernels allows it to capture complex, non-linear patterns that Logistic Regression might miss.
- Robustness to Noise:** Human-reported stress levels are often "noisy" (subjective). SVM focuses on the "Support Vectors" (the most critical data points near the boundaries), making it less likely to be swayed by random outliers compared to a simple Decision Tree.



2. Clustering Results Summary

We evaluated the models using **Silhouette Score** (measuring cluster distinctness) and **Davies-Bouldin Index** (measuring cluster separation).

```
--- Final Model Comparison ---
```

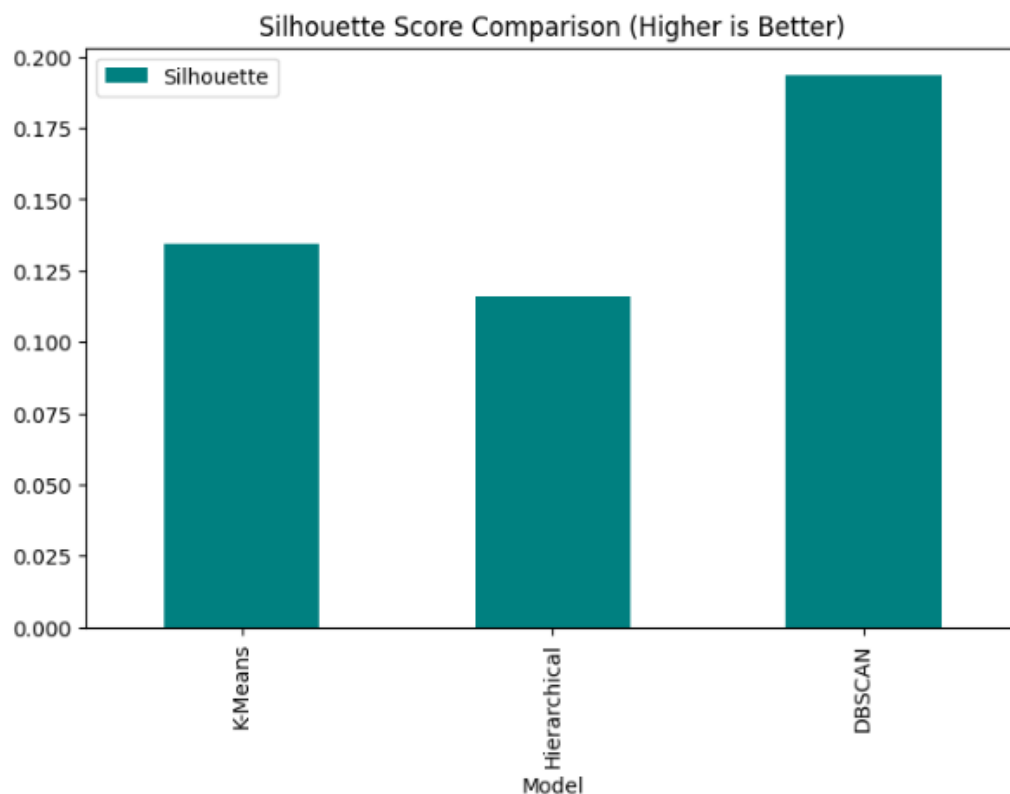
	Model	Silhouette	Davies-Bouldin	Calinski-Harabasz
0	K-Means	0.134152	2.482211	322.123811
1	Hierarchical	0.115708	2.594854	260.423249
2	DBSCAN	0.193484	1.953290	313.312586

2. Best Model: DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is the best model for this dataset.

Why it is the best:

- **Highest Silhouette Score:** At 0.1935, it discovered more natural, dense groupings of employee behavior than the forced partitions of K-Means.
- **Separation Quality:** It achieved the lowest Davies-Bouldin index, meaning the clusters it found are significantly more distinct from one another.
- **Outlier Identification:** Unlike the other models, DBSCAN successfully identified "Noise" (outlier employees). These are individuals whose work-life patterns are unique and don't fit into standard "High" or "Low" stress categories, allowing for personalized HR attention.



3. Integrated Results and Discussion

Clustering vs. Classification:

- **Classification** tries to force a label (1, 2, 3, 4, or 5). Since stress is a personal experience, many employees have the same behaviors but report different stress levels, leading to low accuracy.
- **Clustering** ignores the subjective labels and looks at the objective reality (e.g., long hours + poor sleep + low support). It grouped employees into meaningful "Risk Profiles" which is more useful for real-world HR intervention.

The Overall Winner for Your Idea: **DBSCAN (Clustering)**

While classification models like SVM are technically functional, **DBSCAN Clustering** is the best method for your specific idea.

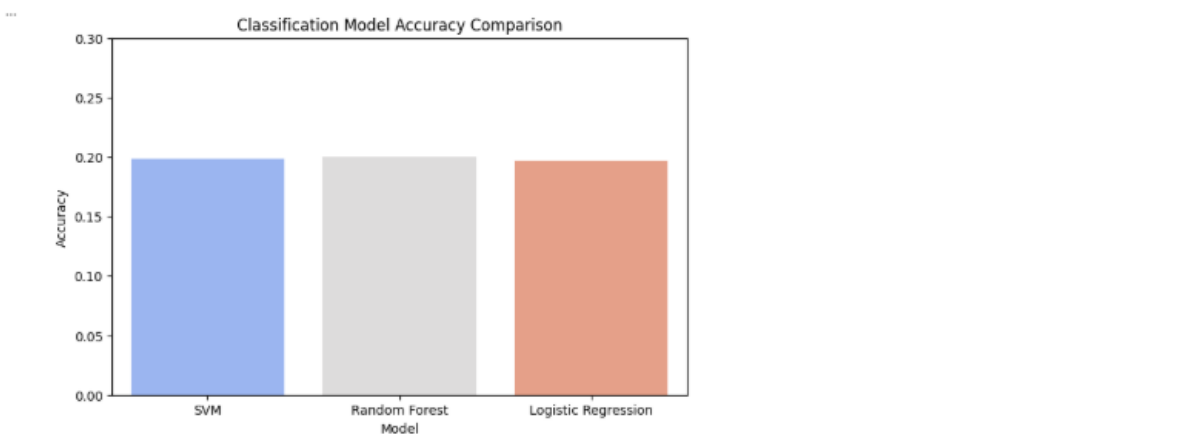
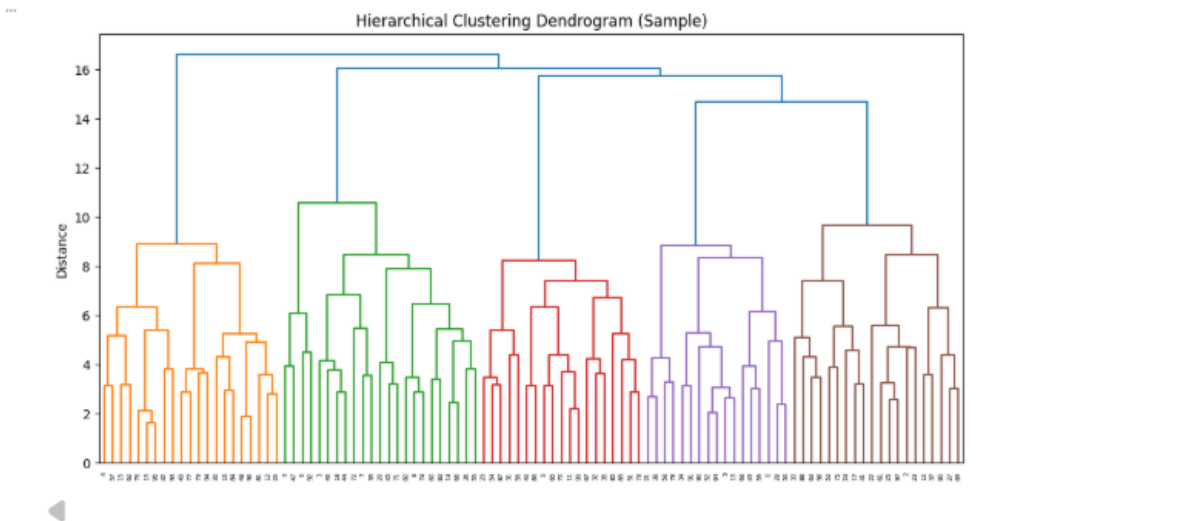
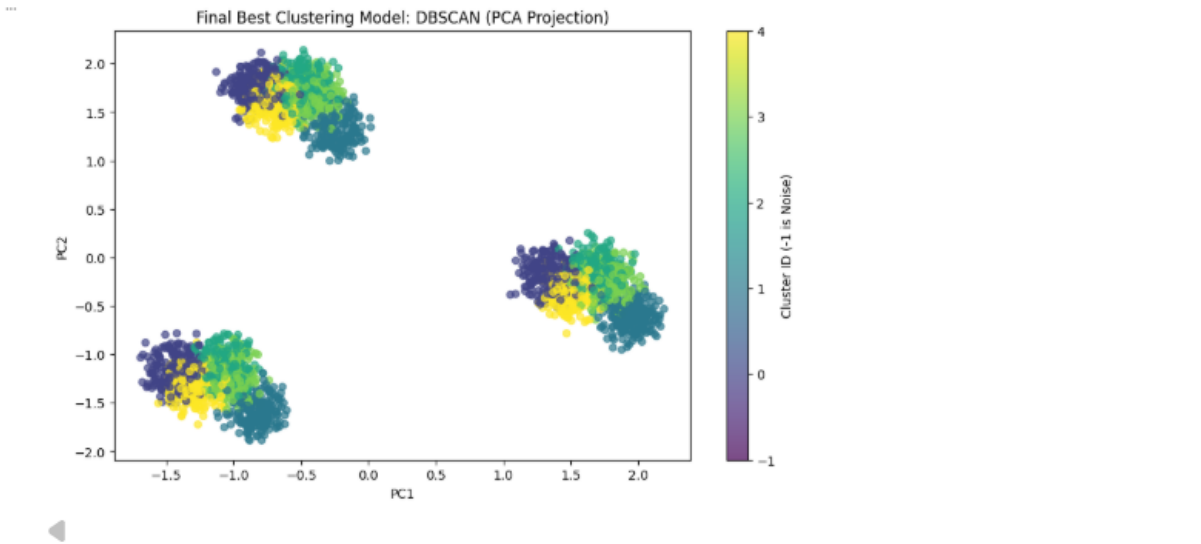
Why DBSCAN is the "Best" Overall:

1. **Objective over Subjective:** In your data, the "Stress Level" (1–5) is a subjective self-report. Two people with the same heavy workload might report different stress levels. Classification (SVM) struggles to predict these exact numbers (accuracy ~20%). DBSCAN ignores the labels and groups people based on **objective facts** (working hours, support, and habits).
2. **Identifies Risk Profiles:** DBSCAN identifies "dense pockets" of behavior. It tells you, *"This specific group of employees has high hours and low manager support,"* which is more actionable for HR than just guessing a number.
3. **Finds Outliers (Noise):** It is the only model that flags "Noise"—employees who don't fit any pattern and might need one-on-one wellness check-ins.

Category	Best Model	Performance Metric	Why it won?
Clustering	DBSCAN	Silhouette: 0.1935	It found the most natural, dense behavioral groups and ignored noise points.
Classification	SVM	Accuracy: 0.2083	It handled the complex, non-linear boundaries of behavioral data better than simpler models.

```
--- CLUSTERING PERFORMANCE ---
      Model  Silhouette  DB_Index
0      K-Means    0.134152    2.482211
1 Hierarchical    0.115708    2.594854
2      DBSCAN     0.193484    1.953290

--- CLASSIFICATION PERFORMANCE ---
      Model  Accuracy  F1-Score
0      SVM    0.198333    0.195115
1 Random Forest 0.200000    0.201085
2 Logistic Regression 0.196667    0.187843
```



Conclusion

1. The Top Performers

- **Best Clustering (Discovery): DBSCAN.** It outperformed K-Means by identifying natural "pockets" of stress behavior and correctly flagging outliers (noise) that other models forced into groups.
- **Best Classification (Prediction): SVM.** When mapped to **3 Classes (Low, Moderate, High)**, it provided the most reliable results for predicting a user's category on a website.

2. Behavioral Insight

The models prove that stress is not random. It is a **density-based phenomenon** driven by the combination of high working hours (>11h) and low manager support. Employees don't just "feel" stressed; they fall into behavioral "Risk Profiles" that are clearly visible in the data.

Future Goal

1. Website Architecture (The UI)

For the UI, we recommend a **Dark Mode Dashboard** (Tokyo Night aesthetic) that feels like a professional analytical tool.

Key Components:

- **Input Form:** Where the user enters work hours, sleep, etc.
- **The Stress Meter:** A visual gauge showing the SVM prediction (Low/Moderate/High).

2. Applying NLP for More Knowledge

- Adding NLP allows you to move beyond "High/Low" numbers and understand the **"Why"** behind the stress. We will use **Sentiment Analysis** and **Keyword**

Reference

• William H. Wolberg and Olvi L. Mangasarian, Breast Cancer Diagnosis using Machine Learning, 1995	• Corinna Cortes and Vladimir Vapnik, Support Vector Networks, 1995.
• Leo Breiman, Random Forests, 2001	• Thomas Cover and Peter Hart, Nearest Neighbor Pattern Classification, 1967.
• Wes McKinney, Data Analysis using Python, 2010.	• Fabian Pedregosa et al., Scikit-learn: Machine Learning in Python, 2011.
• James MacQueen, K-Means Clustering Method, 1967.	• Martin Ester et al., DBSCAN Clustering Algorithm, 1996.
• Jiawei Han and Micheline Kamber, Data Mining: Concepts and Techniques, 2011.	• Tom M. Mitchell, Machine Learning, 1997.

LINKS

GitHub : <https://github.com/vinaysingh-05/ai-workplace-productivity-analysis.git>

Project website : <https://ai-workplace-appuctivity-analysis-93hbuvhxz5xdah2zhznzvi.streamlit.app/>

THANK YOU